

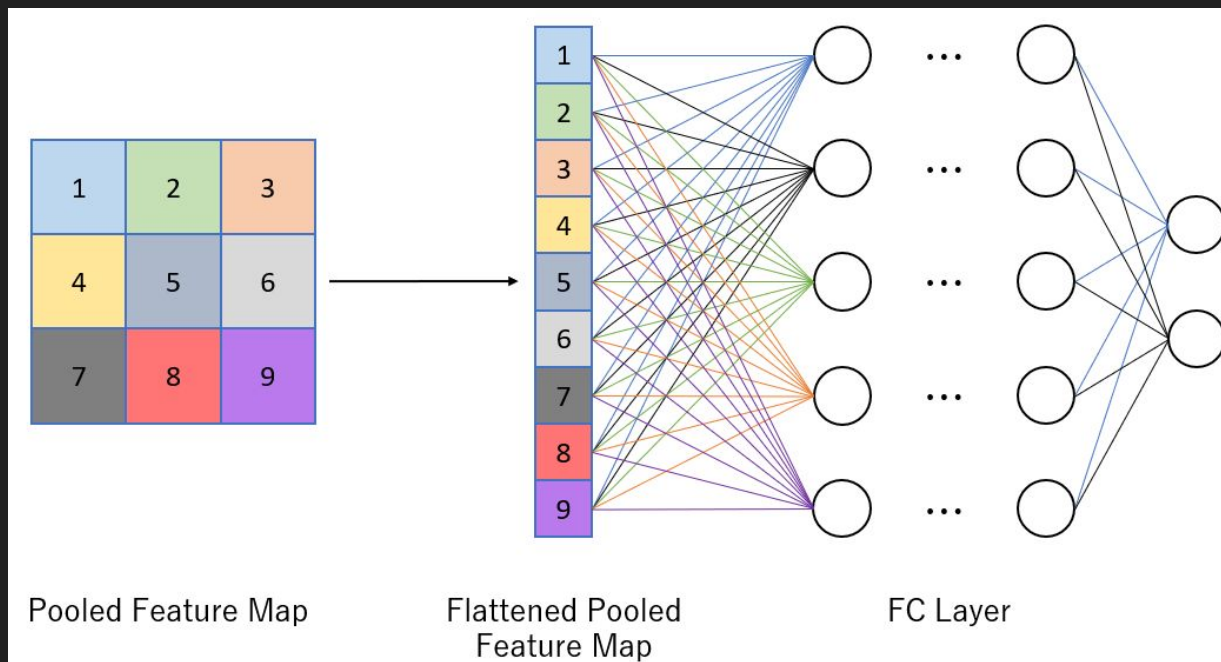
“Теория искусственных нейронных сетей”

Операция свёртки. Архитектуры свёрточных нейронных сетей (CNN)

Петряев Юрий, 24.10.24

Линейный подход для изображений

- Вытягиваем изображение в вектор признаков и применяем к нему линейный слой



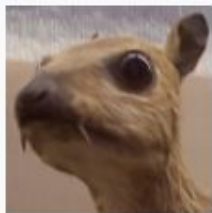
Линейный подход для изображений

- Слишком большое количество параметров:
 - $224 \times 224 \times 3 \approx 200k \Rightarrow$ для простой бинарной классификации необходимо $200k \times 2 = 400k$ параметров
- Анализируем изображение только в фиксированных местах
 - Может решаться аугментациями, но архитектурно никаких допущений про инвариантность к сдвигам не дает

Операция свёртки

Операция свёртки

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

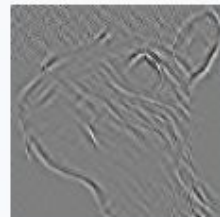


$$\frac{-1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & -476 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

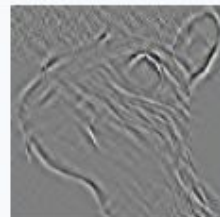


Устранение шума (Sharpen)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

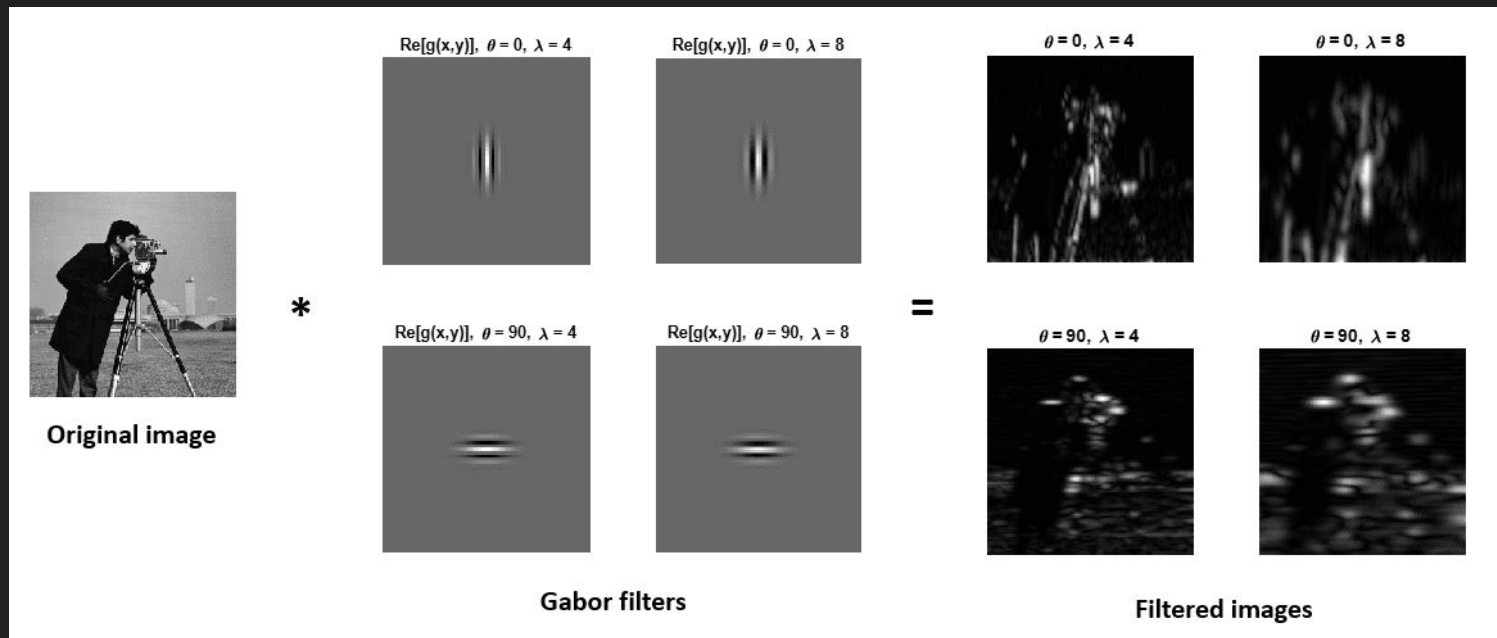


$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Поиск границ (Edge detection)

Операция свёртки

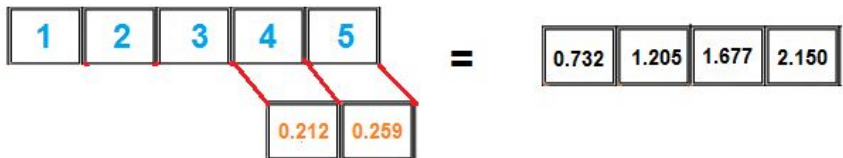
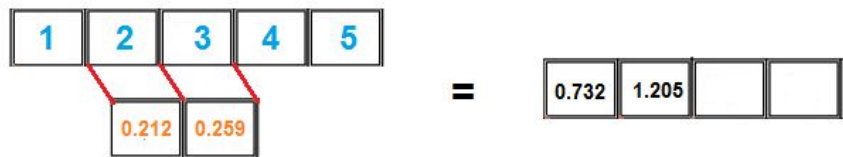


Детекция паттернов (текстур)

Операция свёртки

- **Раньше:** обрабатываем изображения специально построенными свертками
- **Сейчас:** обучаем ядра свертков как параметры нейросети

1D Свёртка



2D Свёртка

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

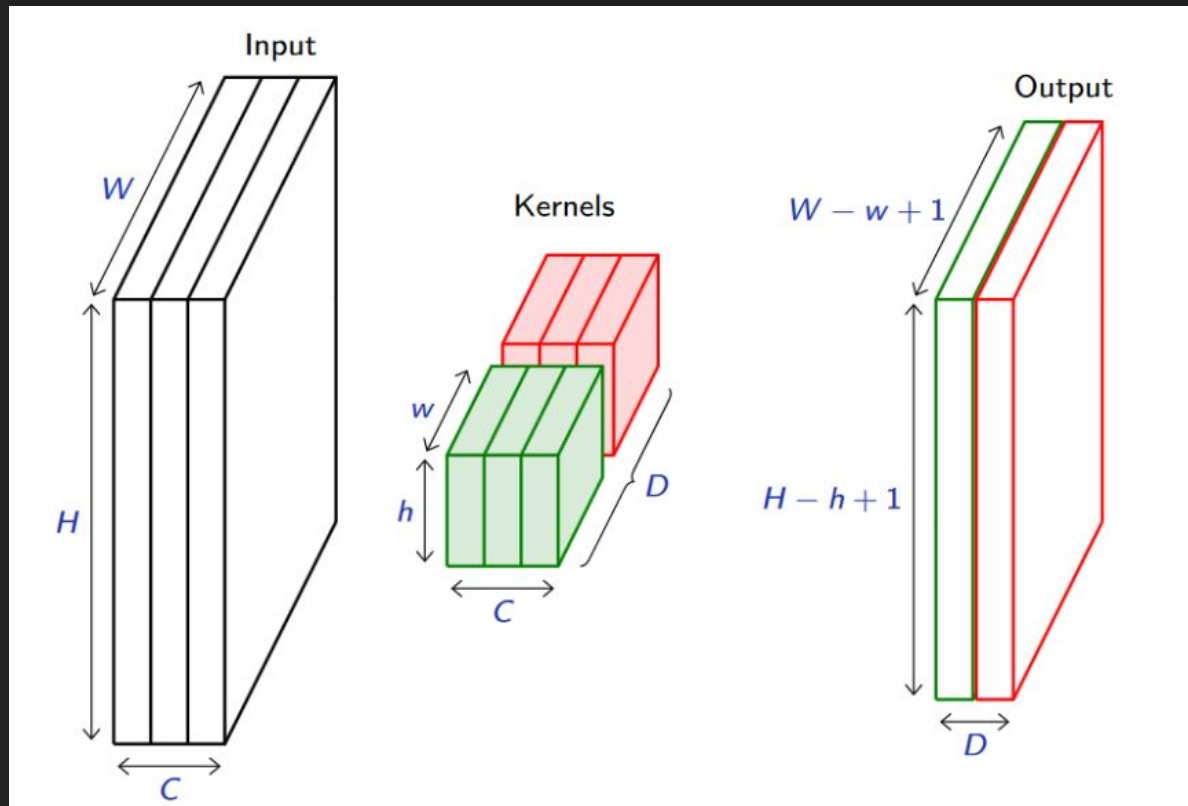
3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

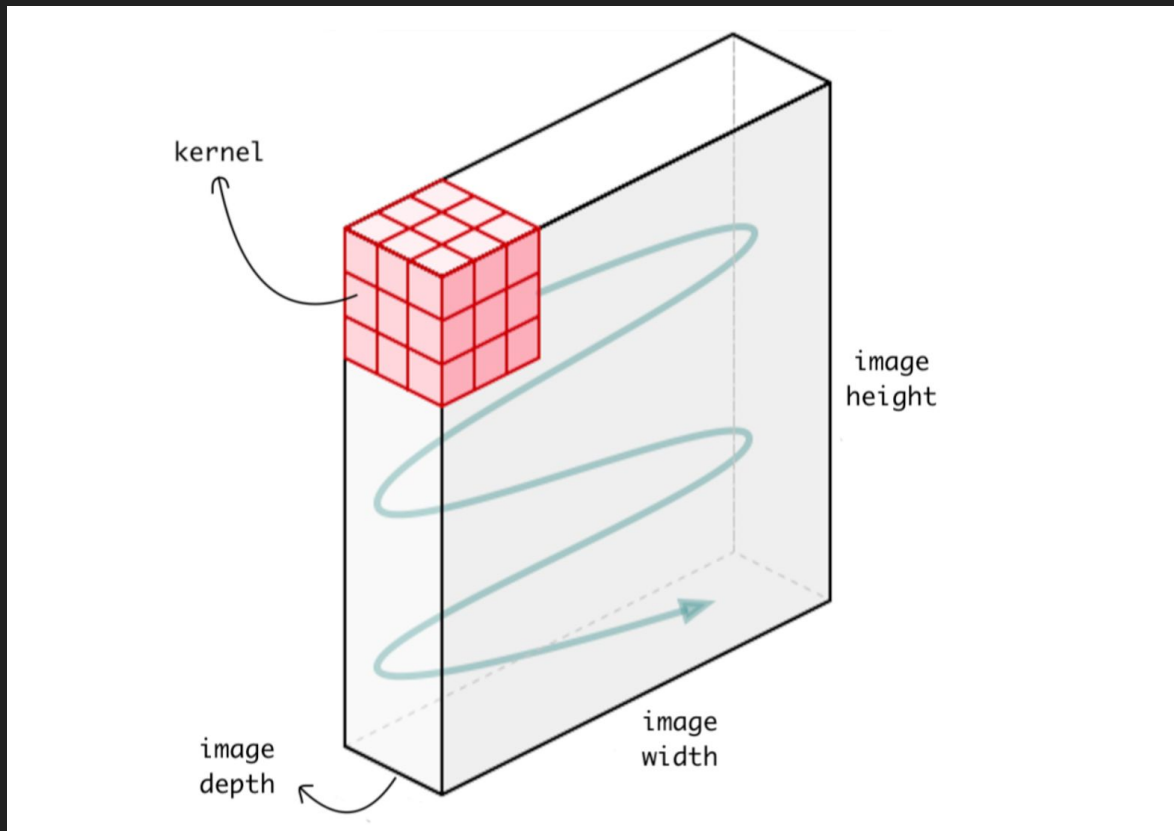
1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

3D Свёртка



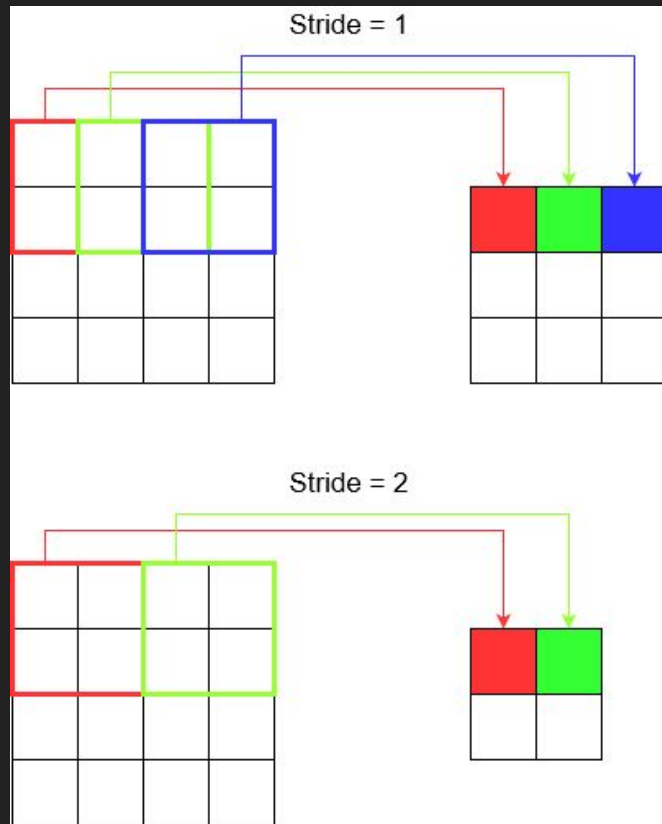
Гиперпараметры свёрток

- **Kernel size**
- **Stride**
- **Padding**
- **Dilation**



Гиперпараметры свёрток: stride

- Регулирует снижение пространственного разрешения
- Признаки получаются менее коррелированными
- По-умолчанию **stride = 1**
- Также распространенный вариант: **stride = kernel size**
 - Такой **stride** использует **pooling**



Гиперпараметры свёрток: padding

- Обычно используется **zeros padding**
- Другие опции (PyTorch):
 - **replicate, reflect, circular**

Replication Padding

5	5	0	8	7	8	1	1
5	5	0	8	7	8	1	1
1	1	9	5	0	7	7	7
6	6	0	2	4	6	6	6
9	9	7	6	6	8	4	4
8	8	3	8	5	1	3	3
7	7	2	7	0	1	0	0
7	7	2	7	0	1	0	0

Reflection Padding

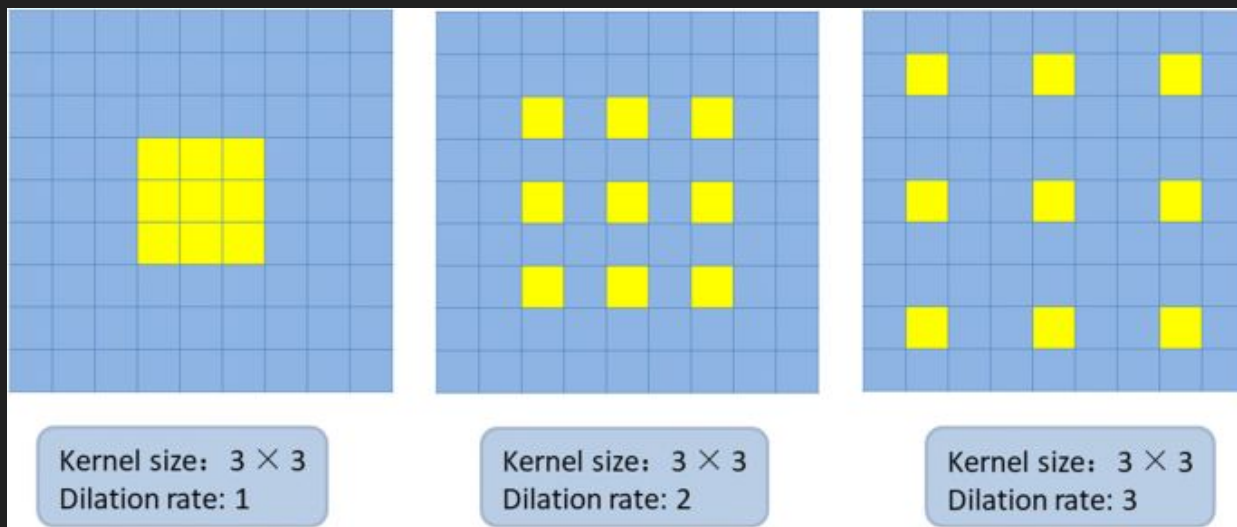
9	1	9	5	0	7	7	7
0	5	0	8	7	8	1	8
9	1	9	5	0	7	7	7
0	6	0	2	4	6	6	6
7	9	7	6	6	8	4	8
3	8	3	8	5	1	3	1
2	7	2	7	0	1	0	1
3	8	3	8	5	1	3	1

Circular Padding

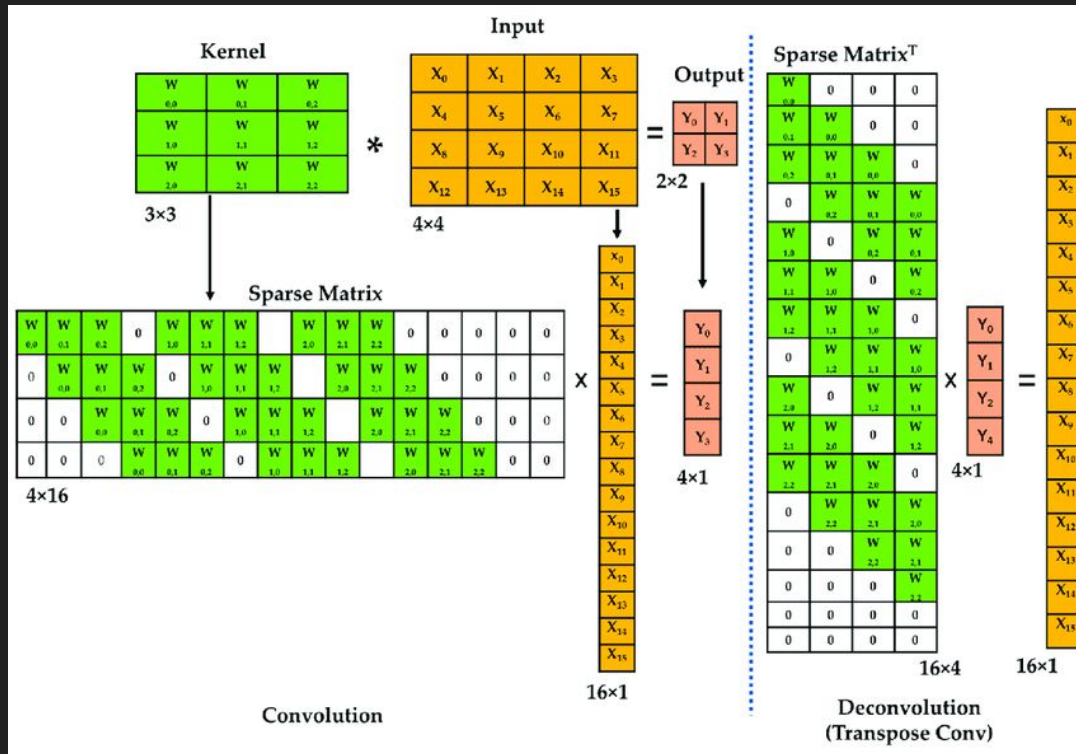
0	7	2	7	0	1	0	7
1	5	0	8	7	8	1	5
7	1	9	5	0	7	7	1
6	6	0	2	4	6	6	6
4	9	7	6	6	8	4	9
3	8	3	8	5	1	3	8
0	7	2	7	0	1	0	7
1	5	0	8	7	8	1	5

Гиперпараметры свёрток: dilation

- Используем ядра “с дырками”
- В отличие от **stride**, покрывает большее **receptive field** с небольшим уменьшением пространственного разрешения

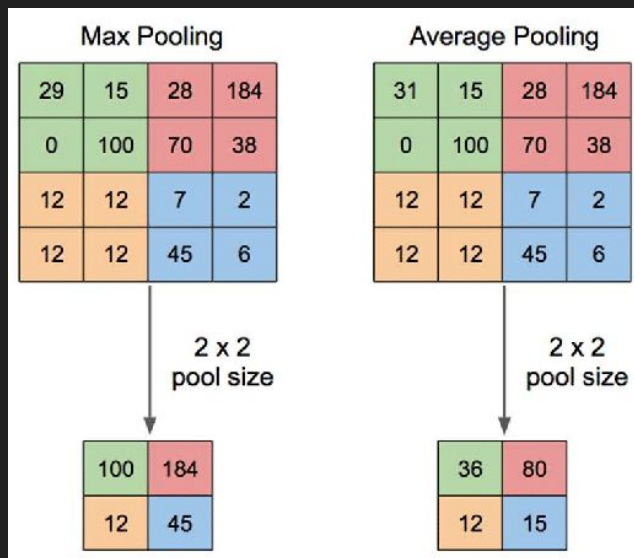


Реализация операции свёртки



Pooling (Downsampling / Subsampling)

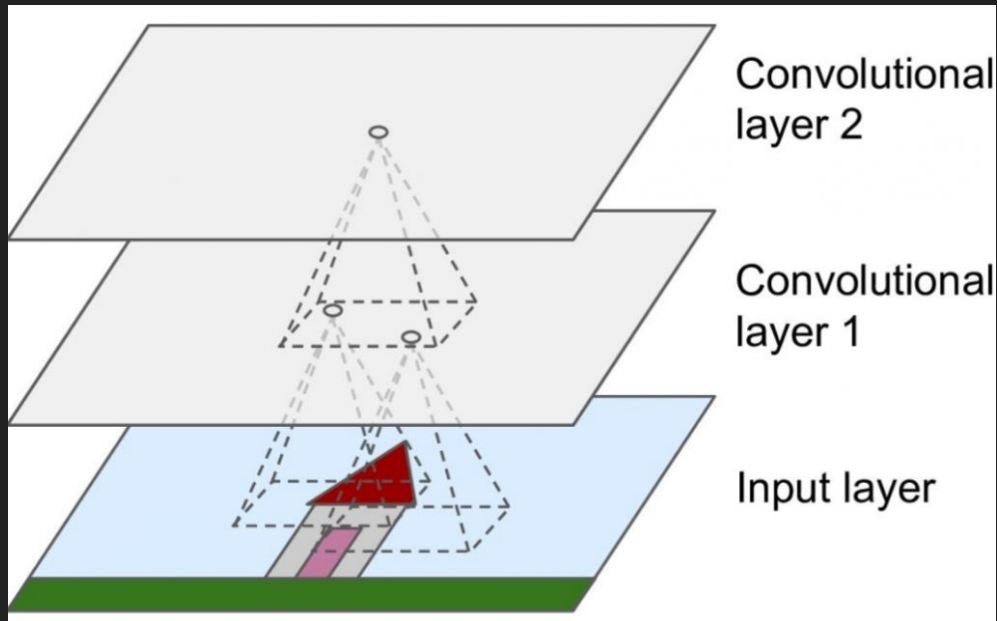
- Гиперпараметры как у свёртки: **kernel size**, **stride**, **padding**, **dilation**
 - По-умолчанию **stride = kernel size**
 - Не имеет обучаемых параметров
- Использовался в старых архитектурах CNN
- Современные архитектуры часто заменяют его на **strided** свёртки



Receptive field

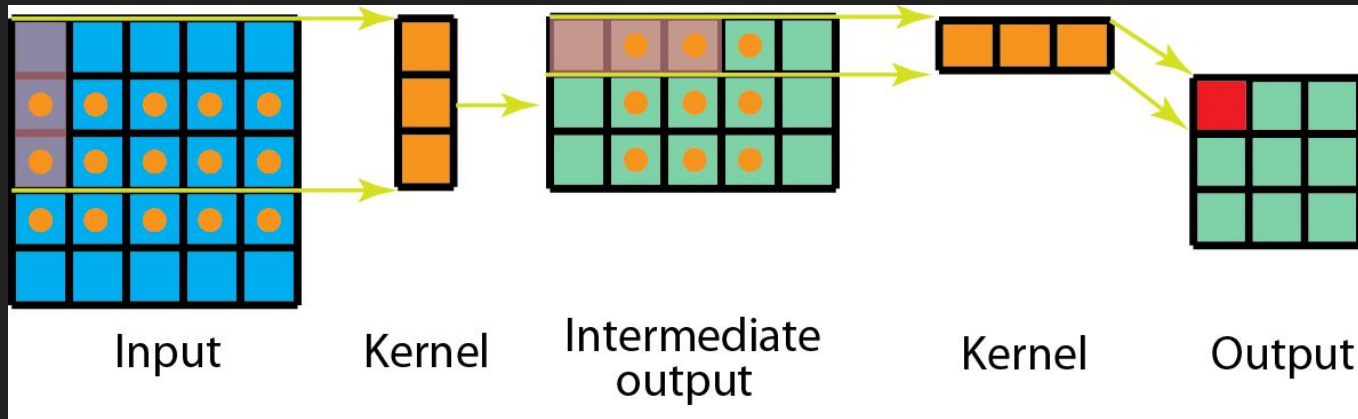
Способы расширения:

- Больше сверточных слоев
- Увеличение **stride**
- Увеличение **dilation**
- Использование **pooling**



Spatially Separable Convolution

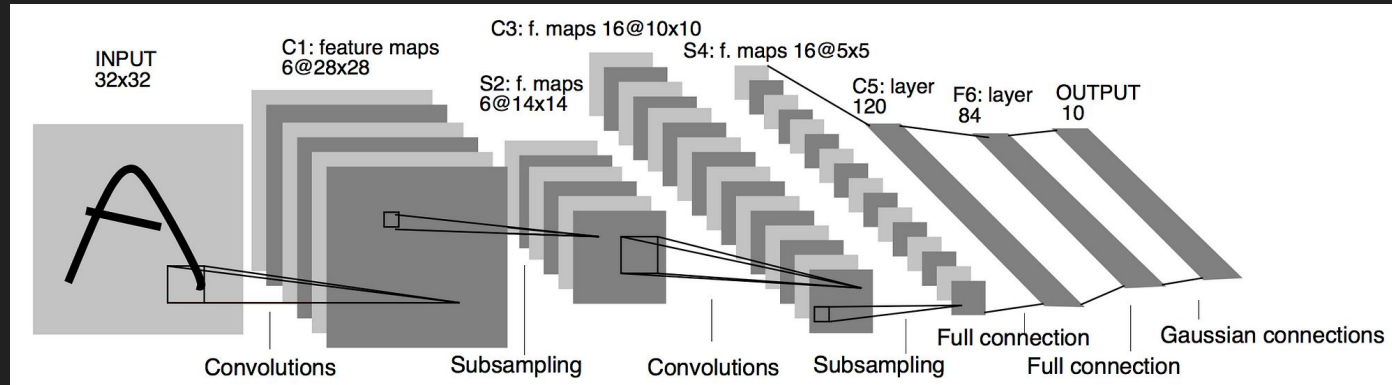
- Факторизуем свёртку
 - Делим ядро 3×3 на ядра 1×3 и 3×1
- Требуется меньше параметров и умножений матриц



Архитектуры CNN

LeNet (1998)

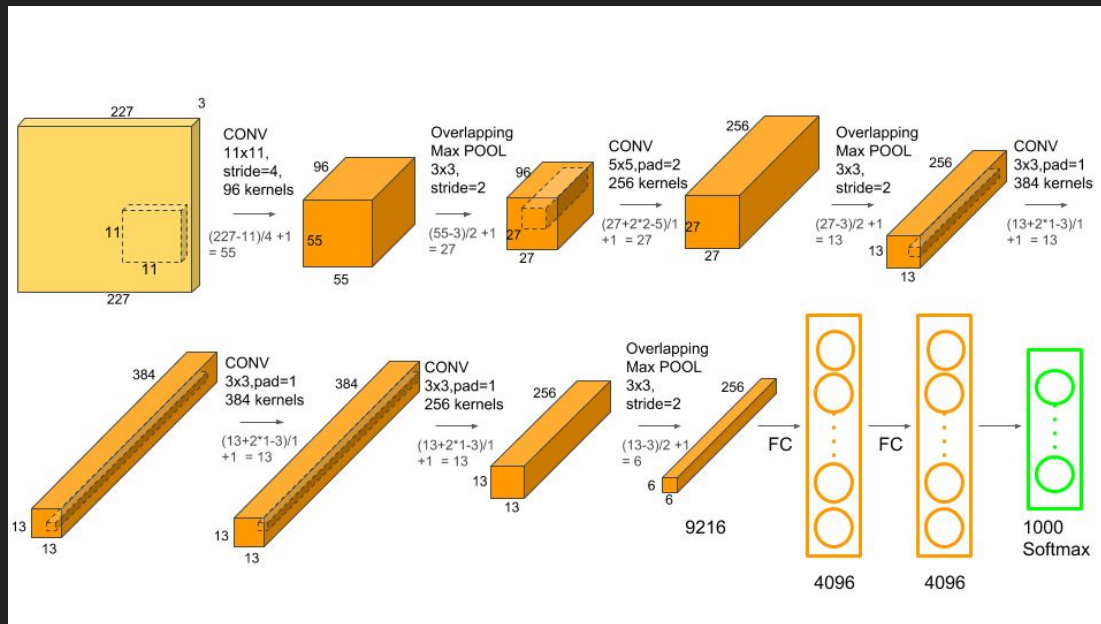
- Два свёрточных слоя:
 - **Convolution (5x5) -> Subsampling (Avg pooling) -> Non-linearity**
- Функция активации – сигмоида
- Голова классификации из трёх линейных слоев (**full connection**)
 - Первый линейный слой по сути свёрточный (ядро 5x5 покрывает карту признаков)
- Размер входного изображения **фиксированный** и зависит от архитектуры



AlexNet (2012)

[Krizhevsky et al., 2012](#)

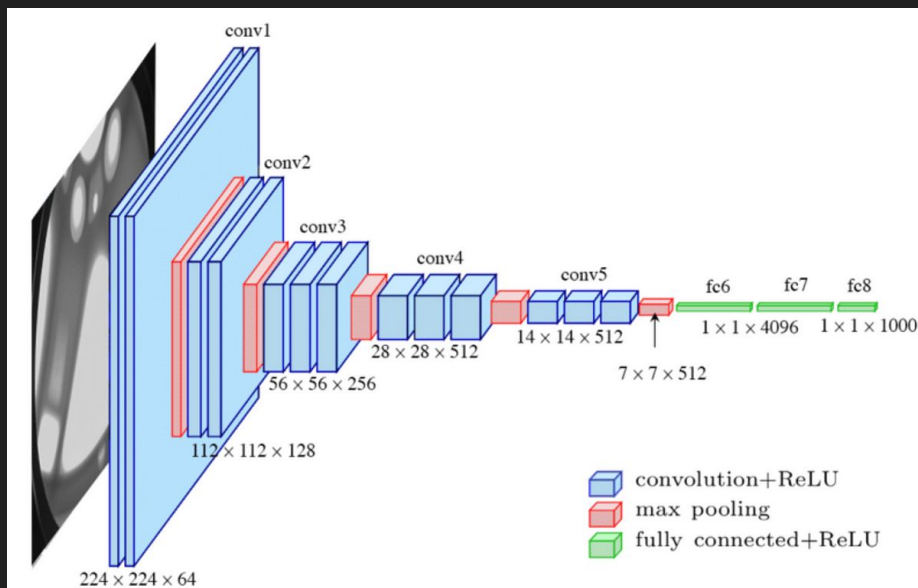
- Пять свёрточных (**CONV + MaxPool / CONV**) и два линейных слоя (**FC**)
- **Max pooling** вместо **Avg pooling**
- **Аугментация данных** и **Dropout** (prob = 0.5) для регуляризации
- Обучалась неделю на двух NVIDIA GTX 580 3GB



VGG (2014)

[Simonyan et al., 2014](#)

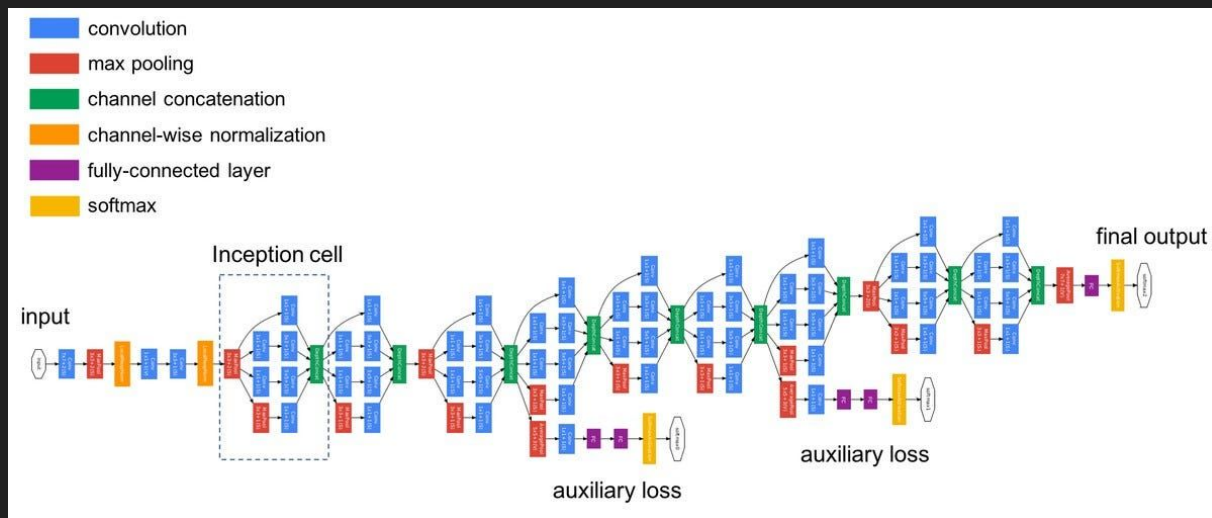
- Пять свёрточных и три линейных слоя
- Каскад 3×3 свёрток вместо 5×5 , 7×7 , 11×11 , ...
- Обучалась 3 недели на 4 GPU
- При обучении end-to-end градиенты затухают
 - Решение авторов – обучать сеть по частям, наращивая глубину



Inception / GoogLeNet (2014)

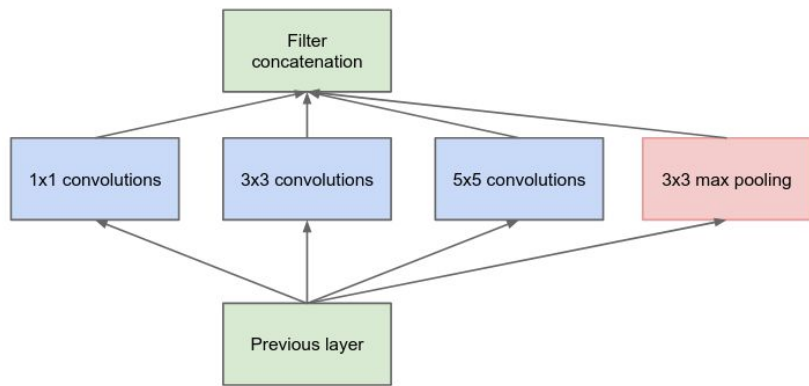
[Szegedy et al., 2014](#)

- “Конструктор” из параллельных **Inception** модулей
- Полностью свёрточная архитектура из 22 слоёв
 - Голова классификации состоит только из одного линейного слоя
- **Global Average Pooling**: не привязываемся к размеру изображения
- Дополнительные классификаторы (**auxiliary loss**) для пробрасывания градиентов

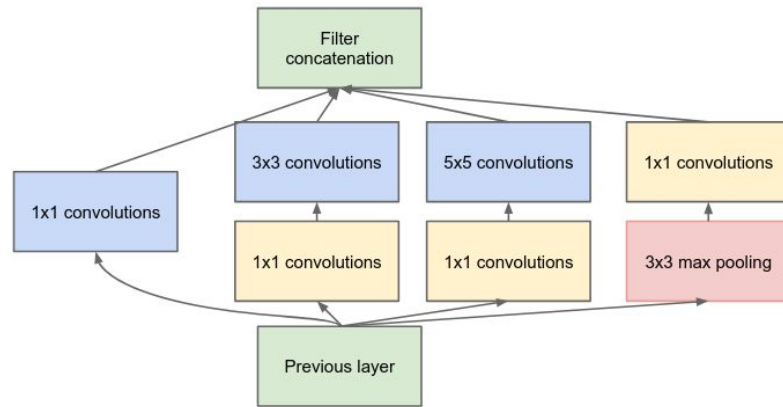


Inception module

- Даем возможность сети самостоятельно “выбирать” тип свёрток
- В целях оптимизации применяются 1×1 свёртки, уменьшающие число каналов
- **Inception v2, v3:**
 - Каскад 3×3 свёрток вместо 5×5
 - Факторизация свёрток



(a) Inception module, naïve version

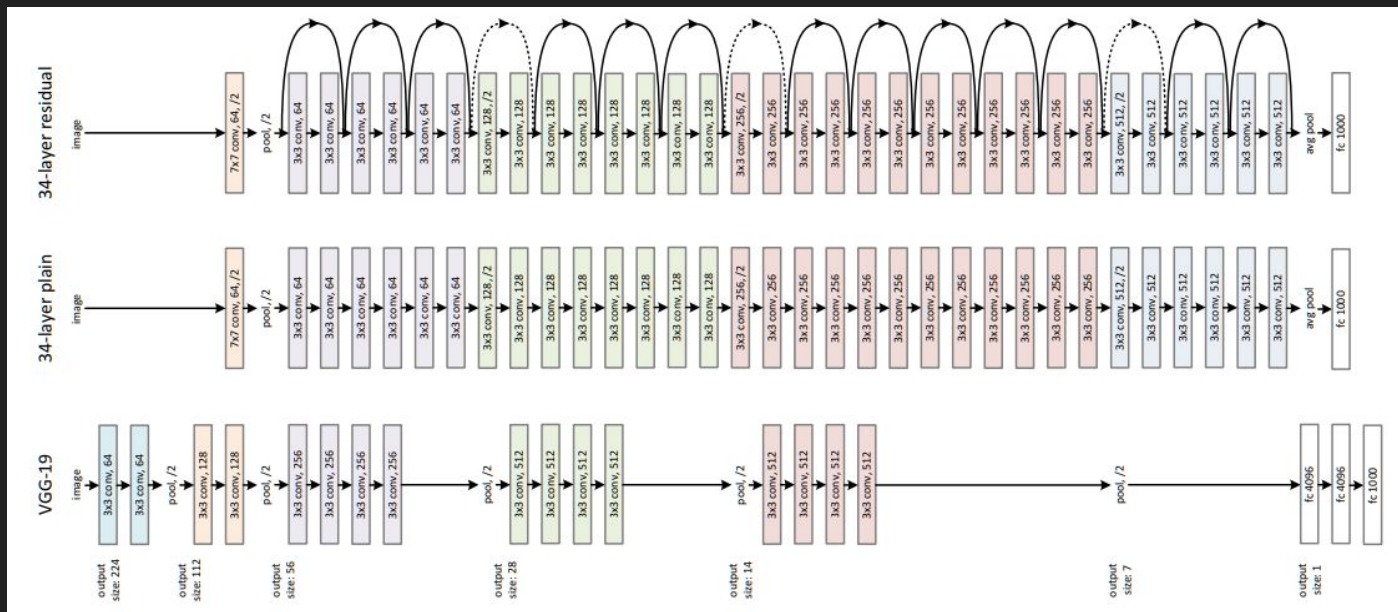


(b) Inception module with dimension reductions

ResNet (2015)

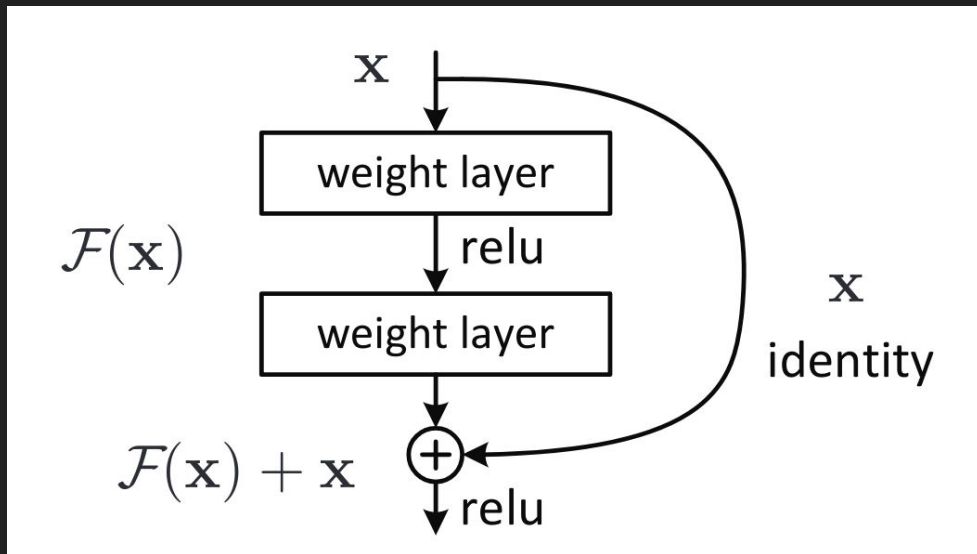
[He et al., 2015](#)

- Вместо дополнительных классификаторов используем **residual connections**
- Меньше каналов, но **больше слоёв** по сравнению с VGG
- **Strided** свёртки вместо **pooling**
- Первое применение **Batch Normalization** перед активациями свёрток
- Отказ от **Dropout**



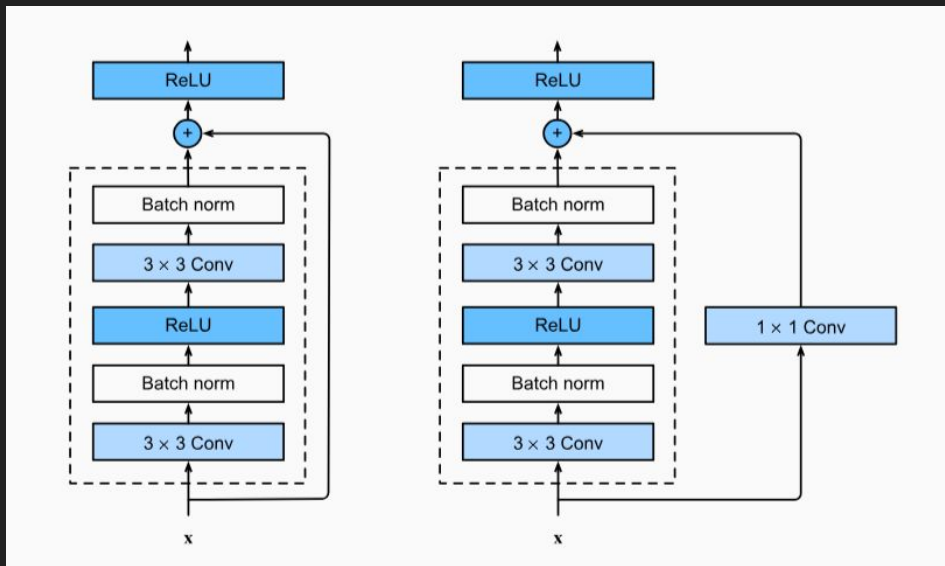
Residual / Skip connections

- Прокидываем связь, по которой градиент будет протекать без изменений
- Обобщение этой идеи – **Highway Networks** ([Srivastava et al., 2015](#))
 - Настраиваем вес связи с помощью небольшой нейросети



Residual block

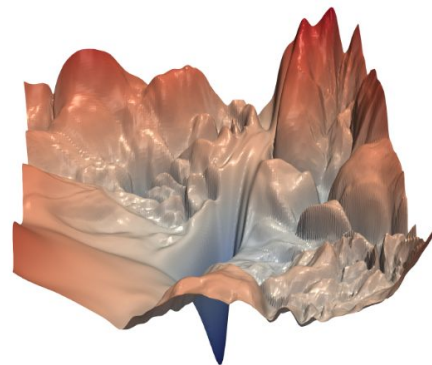
- Может содержать несколько слоёв
- Размерности входа и выхода связи должны совпадать
 - Для этого выполняем линейную проекцию с помощью свёртки 1×1



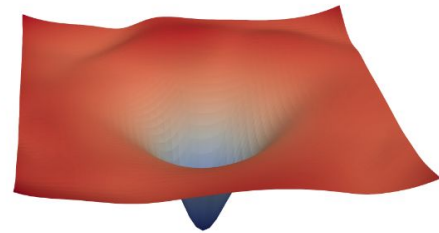
Skip Connection Loss Landscape

[Li et al., 2018](#)

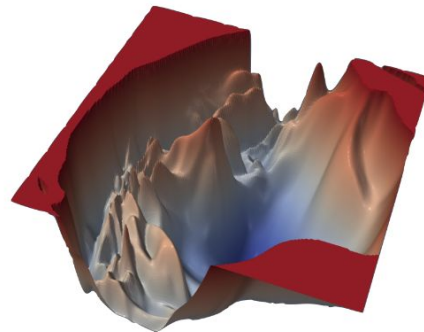
- При увеличении глубины сети **loss surface** становится более хаотичной
- С увеличением ширины сети эффект **обратный**
- **Residual connection** предотвращает переход к хаотичному виду и сглаживает **loss surface**



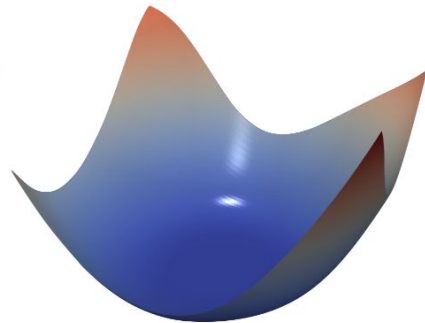
(a) without skip connections



(b) with skip connections



(a) ResNet-110, no skip connections



(b) DenseNet, 121 layers

Семинар

ImageNet

ResNet vs ViT

[Wightman et al., 2021](#)

↓ Architecture	A1-A2-org.		A3		Cost						ImageNet-1k-val				
	train	test	train	test	A1	A2	A1-A2		A3			A1	A2	A3	org.
	res.	res.	res.	res.	time (hour)	# GPU	Pmem	time	# GPU	Pmem	Accuracy(%)				
ResNet-18 [13] [†]	224	224	160	224	186	93	2	12.5	28	2	6.5	71.5	70.6	68.2	69.8
ResNet-34 [13] [†]	224	224	160	224	186	93	2	17.5	27	2	9.0	76.4	75.5	73.0	73.3
ResNet-50 [13] [†]	224	224	160	224	110	55	4	22.0	15	4	11.4	80.4	79.8	78.1	76.1
ResNet-101 [13] [†]	224	224	160	224	74	37	8	16.3	8	8	8.5	81.5	81.3	79.8	77.4
ResNet-152 [13] [†]	224	224	160	224	92	46	8	22.5	9	8	11.8	82.0	81.8	80.6	78.3
RegNetY-4GF [32]	224	224	160	224	130	65	4	27.1	15	4	13.9	81.5	81.3	79.0	79.4
RegNetY-8GF [32]	224	224	160	224	106	53	8	19.8	10	8	10.3	82.2	82.1	81.1	79.9
RegNetY-16GF [32]	224	224	160	224	150	75	8	25.6	13	8	13.4	82.0	82.2	81.7	80.4
RegNetY-32GF [32]	224	224	160	224	120	60	16	17.6	12	16	9.4	82.5	82.4	82.6	81.0
SE-ResNet-50 [20]	224	224	160	224	102	51	4	27.6	16	4	14.2	80.0	80.1	77.0	76.7
SENet-154 [20]	224	224	160	224	110	55	16	23.3	12	16	12.2	81.7	81.8	81.9	81.3
ResNet-50-D [14]	224	224	160	224	100	50	4	23.9	14	4	12.3	80.7	80.2	78.7	79.3
ResNeXt-50-32x4d [51] [†]	224	224	160	224	80	40	8	14.3	15	4	14.6	80.5	80.4	79.2	77.6
EfficientNet-B0 [41]	224	224	160	224	110	55	4	22.1	15	4	11.4	77.0	76.8	73.0	77.1
EfficientNet-B1 [41]	240	240	160	224	62	31	8	17.9	8	8	7.9	79.2	79.4	74.9	79.1
EfficientNet-B2 [41]	260	260	192	256	76	38	8	22.8	9	8	11.9	80.4	80.1	77.5	80.1
EfficientNet-B3 [41]	300	300	224	288	62	31	16	19.5	6	16	10.1	81.4	81.4	79.2	81.6
EfficientNet-B4 [41]	380	380	320	380	64	32	32	20.4	8	32	14.3	81.6	82.4	81.2	82.9
ViT-Ti [45]*	224	224	160	224	98	49	4	16.3	14	4	7.0	74.7	74.1	66.7	72.2
ViT-S [45]*	224	224	160	224	68	34	8	16.1	8	8	7.0	80.6	79.6	73.8	79.8
ViT-B [11]*	224	224	160	224	66	33	16	16.4	5	16	7.3	80.4	79.8	76.0	81.8

ConvNet vs ViT

[Liu et al., 2022](#)

[Yu et al., 2024](#)

- **ConvNeXt** – имитируем архитектуру трансформера с помощью **ResNet**
- **InceptionNeXt** – используем идею **Inception**, чтобы ускорить **ConvNeXt** с сохранением качества

Аугментации изображений

Transfer Learning